

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng nền tảng thương mại điện tử đa thuê bao
với kiến trúc Zero-file và Dynamic Rendering**

Trịnh Vi Giang Xuân

xuan.tvg225118@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin

Giảng viên hướng dẫn: PGS. TS. Phạm Văn ABC

Chữ kí GVHD

Khoa:

Khoa học máy tính

Trường:

Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2022

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành nhất đến TS. Nguyễn Quốc Tuấn. Thầy đã tận tình hướng dẫn, định hướng phát triển và chỉ bảo cặn kẽ, giúp em hoàn thành đồ án tốt nghiệp này.

Em cũng xin trân trọng cảm ơn Ban Giám hiệu Trường Công nghệ Thông tin và Truyền thông cùng quý thầy cô trong trường. Những kiến thức nền tảng quý giá mà các thầy cô truyền đạt chính là cơ sở vững chắc để em thực hiện đề tài. Bên cạnh đó, em xin tri ân gia đình, bạn bè đã luôn động viên, hỗ trợ và tạo mọi điều kiện thuận lợi cho em trong suốt quá trình học tập.

Cuối cùng, em xin cam đoan các kết quả trong báo cáo là thành quả trung thực của bản thân. Dù đã nỗ lực, đồ án vẫn khó tránh khỏi thiếu sót do giới hạn về kiến thức và kinh nghiệm. Em rất mong nhận được sự góp ý từ quý thầy cô để đề tài được hoàn thiện hơn.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh thương mại điện tử tại Việt Nam đang phát triển mạnh mẽ, các doanh nghiệp vừa và nhỏ thường xuyên đối mặt với rào cản lớn về mặt chi phí cũng như rào cản kỹ thuật khi muốn xây dựng và vận hành một cửa hàng trực tuyến độc lập. Các nền tảng thương mại điện tử hiện có trên thị trường như Shopify, WooCommerce hay Haravan vẫn tồn tại nhiều hạn chế đáng kể, đặc biệt là trong việc không thể đáp ứng đồng thời cả ba tiêu chí: chi phí khởi tạo và duy trì thấp, khả năng tùy biến giao diện linh hoạt mà không yêu cầu kiến thức lập trình, và khả năng mở rộng hệ thống để phục vụ hàng chục ngàn cửa hàng mà không làm gia tăng tuyến tính chi phí hạ tầng. Xuất phát từ thực tiễn đó, đồ án này tập trung vào việc nghiên cứu và phát triển nền tảng thương mại điện tử đa thuê bao mang tên ShopVolo v2 nhằm giải quyết triệt để bài toán tối ưu hóa nguồn lực. Hướng tiếp cận chủ đạo của nghiên cứu là áp dụng kiến trúc phần mềm Multi-tenant SaaS kết hợp với nguyên lý thiết kế "Zero-file", trong đó toàn bộ giao diện của các cửa hàng được biểu diễn dưới dạng dữ liệu cấu hình thay vì mã nguồn vật lý riêng biệt.

Hệ thống được phát triển dựa trên mô hình Monorepo với công cụ Turborepo, tích hợp các nền tảng công nghệ hiện đại bao gồm Next.js cho giao diện trang quản trị và kết xuất mặt tiền cửa hàng động, NestJS cho hệ thống API lõi với khả năng cô lập dữ liệu người dùng tự động, cùng hệ thống cơ sở dữ liệu kép kết hợp giữa PostgreSQL và MongoDB để xử lý hiệu quả cả thông tin có cấu trúc lẫn tài liệu cấu hình linh hoạt. Điểm nhấn của kiến trúc hệ thống là thuật toán Deep Merge [1], cho phép trộn lẫn cấu trúc giao diện chuẩn và cấu hình tùy chỉnh của từng cá nhân theo thời gian thực, kết hợp với bộ nhớ đệm Redis để đảm bảo tốc độ phản hồi gần như tức thời. [TODO: Bổ sung chi tiết về những đóng góp chính và các kết quả hiệu năng cụ thể đạt được sau quá trình triển khai, kiểm thử hệ thống].

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

In the context of the rapidly growing e-commerce sector in Vietnam, small and medium-sized enterprises (SMEs) frequently encounter significant financial and technical barriers when attempting to build and operate independent online stores. Existing e-commerce platforms on the market, such as Shopify, WooCommerce, or Haravan, still exhibit considerable limitations. Specifically, they fail to simultaneously satisfy three crucial criteria: low initial and maintenance costs, flexible interface customization without requiring programming knowledge, and system scalability capable of serving tens of thousands of stores without linearly increasing infrastructure costs. Stemming from this practical need, this thesis focuses on the research and development of a multi-tenant e-commerce platform named ShopVolo v2 to thoroughly address the resource optimization problem. The primary approach of the study involves implementing a Multi-tenant Software as a Service (SaaS) architecture combined with the "Zero-file" design principle, wherein the entire storefront interface is represented as configuration data rather than discrete physical source code files.

The system is developed based on a Monorepo model using the Turborepo tool, integrating modern technology stacks including Next.js for the administrative dashboard interface and dynamic storefront rendering, and NestJS for the core API system with automatic user data isolation capabilities. Furthermore, a dual database system combining PostgreSQL and MongoDB is employed to efficiently process both structured information and flexible configuration documents. The highlight of the system architecture is the Deep Merge algorithm, which allows the real-time blending of the standard interface structure with individual custom configurations, combined with Redis caching to ensure near-instantaneous response speeds. [TODO: Supplement details regarding the main contributions and specific performance results achieved after the deployment and testing of the system].

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.2 Tổng quan chức năng	6
2.2.1 Biểu đồ use case tổng quát	7
2.2.2 Biểu đồ use case phân rã — Quản lý Shop.....	7
2.2.3 Biểu đồ use case phân rã — Quản lý Sản phẩm & Đơn hàng	8
2.2.4 Biểu đồ use case phân rã — Trải nghiệm Khách hàng.....	8
2.2.5 Quy trình nghiệp vụ — Onboarding và Dynamic Rendering	9
2.3 Đặc tả chức năng	9
2.3.1 Đặc tả use case: Khởi tạo Shop (UC-01).....	10
2.3.2 Đặc tả use case: Tùy biến giao diện (UC-03).....	11
2.3.3 Đặc tả use case: Quản lý sản phẩm (UC-04)	12
2.3.4 Đặc tả use case: Duyệt sản phẩm động (UC-07).....	13
2.3.5 Đặc tả use case: Thanh toán và Đơn hàng (UC-09).....	14
2.4 Yêu cầu phi chức năng	14
CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG.....	17
3.1 Turborepo — Quản lý Monorepo	17
3.2 NestJS — Framework Backend	17
3.3 Next.js — Framework Frontend	18

3.4 PostgreSQL và Prisma ORM — Cơ sở dữ liệu quan hệ	18
3.5 MongoDB và Mongoose — Cơ sở dữ liệu tài liệu	18
3.6 Redis — Cache In-memory	19
3.7 MinIO — Object Storage.....	19
3.8 Better Auth — Xác thực và Phân quyền	20
3.9 Docker và Kubernetes — Containerization và Orchestration.....	20
CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	22
4.1 Thiết kế kiến trúc.....	22
4.1.1 Lựa chọn kiến trúc phần mềm	22
4.1.2 Thiết kế tổng quan.....	22
4.1.3 Thiết kế chi tiết gói	22
4.2 Thiết kế chi tiết.....	23
4.2.1 Thiết kế giao diện	23
4.2.2 Thiết kế lớp	23
4.2.3 Thiết kế cơ sở dữ liệu	24
4.3 Xây dựng ứng dụng.....	24
4.3.1 Thư viện và công cụ sử dụng.....	24
4.3.2 Kết quả đạt được	24
4.3.3 Minh họa các chức năng chính	24
4.4 Kiểm thử.....	24
4.5 Triển khai	25
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	26
5.1 Thuật toán Deep Merge trong kiến trúc Zero-file	26
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	29
6.1 Kết luận	29
6.2 Hướng phát triển.....	29

TÀI LIỆU THAM KHẢO.....	33
PHỤ LỤC.....	35
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP	35
A.1 Ngành học.....	36
A.2 Đánh dấu (bullet) và đánh số (numering)	36
A.3 Cách thêm bảng	37
A.4 Chèn hình ảnh	37
A.5 Tài liệu tham khảo	38
A.6 Cách viết phương trình và công thức toán học.....	38
A.7 Qui cách đóng quyển.....	38
B. ĐẶC TẢ USE CASE.....	40
B.1 Đặc tả use case “Thống kê tình hình mượn sách”	40
B.2 Đặc tả use case “Đăng ký làm thẻ mượn”	40

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát hệ thống ShopVolo v2	7
Hình 2.2	Biểu đồ use case phân rã nhóm Quản lý Shop	7
Hình 2.3	Biểu đồ use case phân rã nhóm Sản phẩm và Đơn hàng	8
Hình 2.4	Biểu đồ use case phân rã nhóm trải nghiệm Khách hàng	8
Hình 2.5	Biểu đồ hoạt động quy trình Onboarding và Rendering động	9
Hình 2.6	Biểu đồ hoạt động luồng tải lên và xử lý ảnh sản phẩm	12
Hình 2.7	Biểu đồ hoạt động quy trình thanh toán và xác nhận đơn hàng	14
Hình 4.1	Ví dụ biểu đồ phụ thuộc gói	22
Hình 4.2	Ví dụ thiết kế gói	23
Hình 5.1	Biểu đồ luồng hoạt động thuật toán Deep Merge	28
Hình A.1	Internet vạn vật	37
Hình A.2	Quy cách đóng quyển đồ án	39
Hình A.3	Quy cách đóng quyển đồ án	39

DANH MỤC BẢNG BIỂU

Bảng 1.1	So sánh các nền tảng thương mại điện tử	2
Bảng 2.1	So sánh chi tiết các nền tảng thương mại điện tử	6
Bảng 2.2	Đặc tả use case Khởi tạo Shop (UC-01)	10
Bảng 2.3	Đặc tả use case Tùy biến thương hiệu (UC-03)	11
Bảng 2.4	Đặc tả use case Quản lý sản phẩm (UC-04)	12
Bảng 2.5	Đặc tả use case Duyệt sản phẩm động (UC-07)	13
Bảng 2.6	Đặc tả use case Thanh toán và Đơn hàng (UC-09)	14
Bảng 3.1	Tổng hợp công nghệ sử dụng trong hệ thống ShopVolo v2 . .	21
Bảng 4.1	Danh sách thư viện và công cụ sử dụng	24
Bảng A.1	Table to test captions and labels.	37

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
ACID	Tính nguyên tử, nhất quán, độc lập và bền vững (Atomicity, Consistency, Isolation, Durability)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
CDN	Mạng phân phối nội dung (Content Delivery Network)
CI/CD	Tích hợp liên tục và Triển khai liên tục (Continuous Integration/Continuous Deployment)
CRUD	Tạo, Đọc, Cập nhật, Xóa (Create, Read, Update, Delete)
DAG	Đồ thị có hướng phi chu trình (Directed Acyclic Graph)
DDL	Ngôn ngữ định nghĩa dữ liệu (Data Definition Language)
ER	Thực thể - Kết hợp (Entity-Relationship)
EUD	Phát triển ứng dụng người dùng cuối (End-User Development)
GWT	Công cụ lập trình Javascript bằng Java của Google (Google Web Toolkit)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
IaaS	Dịch vụ hạ tầng
ISR	Tái tạo tĩnh tăng dần (Incremental Static Regeneration)
JSON	JavaScript Object Notation
JWT	JSON Web Token
ODM	Ánh xạ tài liệu đối tượng (Object-Document Mapping)

Thuật ngữ	Ý nghĩa
ORM	Ánh xạ quan hệ đối tượng (Object-Relational Mapping)
RBAC	Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control)
REST	Representational State Transfer
S3	Dịch vụ lưu trữ đơn giản (Simple Storage Service)
SaaS	Software as a Service — Phần mềm dịch vụ
SME	Doanh nghiệp vừa và nhỏ (Small and Medium Enterprise)
SSR	Kết xuất phía máy chủ (Server-Side Rendering)
TTL	Thời gian sống (Time To Live)
UC	Trường hợp sử dụng (Use Case)
UI	Giao diện người dùng (User Interface)
UML	Ngôn ngữ mô hình hóa thống nhất (Unified Modeling Language)
UX	Trải nghiệm người dùng (User Experience)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này giới thiệu tổng quan về bối cảnh và mục tiêu của đề tài nghiên cứu. Đầu tiên, phần 1.1 đặt vấn đề từ thực tiễn nhu cầu xây dựng cửa hàng trực tuyến của các doanh nghiệp vừa và nhỏ tại Việt Nam. Tiếp đó, phần 1.2 phân tích các giải pháp hiện có trên thị trường để làm rõ những hạn chế và xác định các mục tiêu cụ thể của hệ thống đề xuất. Phần 1.3 trình bày ngắn gọn định hướng kiến trúc đa thuê bao và công nghệ được lựa chọn để giải quyết bài toán. Cuối cùng, phần 1.4 mô tả bố cục chi tiết của toàn bộ báo cáo đồ án tốt nghiệp này.

1.1 Đặt vấn đề

Thương mại điện tử tại Việt Nam đang trải qua giai đoạn tăng trưởng mạnh mẽ trong những năm gần đây [2]. Sự chuyển dịch thói quen mua sắm của người tiêu dùng sang các nền tảng trực tuyến đã thúc đẩy các hộ kinh doanh và doanh nghiệp vừa và nhỏ liên tục tìm kiếm cơ hội mở rộng kênh bán hàng. Nhu cầu sở hữu một cửa hàng trực tuyến riêng biệt, chuyên nghiệp để xây dựng thương hiệu đang ngày càng trở nên cấp thiết.

Tuy nhiên, để xây dựng và vận hành một cửa hàng trực tuyến độc lập, các doanh nghiệp vừa và nhỏ phải đối mặt với nhiều rào cản lớn. Chi phí ban đầu cho việc thiết kế và phát triển website thường dao động từ mười đến năm mươi triệu đồng. Bên cạnh đó, việc duy trì một đội ngũ kỹ thuật để quản lý máy chủ, bảo mật và cập nhật tính năng liên tục là một gánh nặng tài chính không nhỏ. Hơn nữa, thời gian triển khai từ khi lên ý tưởng đến khi ra mắt thường kéo dài từ một đến ba tháng, làm chậm khả năng phản ứng với thị trường.

Các nền tảng phần mềm dưới dạng dịch vụ hiện có trên thị trường vẫn tồn tại nhiều hạn chế đáng kể. Các nền tảng quốc tế thường có chi phí duy trì hàng tháng cao, không tối ưu cho các phương thức thanh toán địa phương và giới hạn khả năng can thiệp sâu. Các giải pháp tự triển khai lại đòi hỏi người sử dụng phải có kiến thức kỹ thuật chuyên sâu để tự thiết lập hạ tầng và bảo mật. Trong khi đó, các nền tảng trong nước tuy dễ tiếp cận nhưng lại thiếu đi khả năng cho phép người dùng tự do tùy biến giao diện một cách sâu rộng mà không cần viết mã. Hiện tại, chưa có giải pháp nào đáp ứng đồng thời cả ba tiêu chí: chi phí thấp, khả năng tùy biến cao, và khả năng vận hành hàng chục ngàn cửa hàng mà không làm tăng tuyến tính chi phí hạ tầng.

Một nền tảng phần mềm mở, cho phép các doanh nghiệp vừa và nhỏ tại Việt Nam khởi động cửa hàng trực tuyến chỉ trong vài phút mà không cần kỹ năng lập

trình sẽ là một bước tiến quan trọng. Giải pháp này không chỉ giúp tiết kiệm chi phí và thời gian triển khai, mà còn cung cấp một giao diện chuyên nghiệp, góp phần đẩy nhanh quá trình số hóa thương mại.

1.2 Mục tiêu và phạm vi đề tài

Trên thị trường hiện nay, có nhiều nền tảng cung cấp giải pháp xây dựng cửa hàng trực tuyến với các đặc điểm khác nhau. Shopify là một trong những nền tảng cung cấp phần mềm dưới dạng dịch vụ hàng đầu, cung cấp giao diện đẹp mắt nhưng có chi phí duy trì khá cao từ 29\$ đến 299\$ mỗi tháng [3]. Hơn nữa, nền tảng này cũng giới hạn khả năng tùy biến sâu vào mã nguồn giao diện, và chưa thực sự phù hợp với các doanh nghiệp nhỏ tại Việt Nam về mặt chi phí và hỗ trợ thanh toán nội địa.

WooCommerce là một thành phần mở rộng mã nguồn mở dành cho hệ thống quản trị nội dung WordPress, cho phép tạo cửa hàng trực tuyến hoàn toàn miễn phí [4]. Mặc dù mang lại khả năng tùy biến sâu, giải pháp này lại yêu cầu người sử dụng phải tự thuê máy chủ lưu trữ, tự cấu hình bảo mật và thường xuyên thực hiện các bản cập nhật. Việc này đòi hỏi trình độ kỹ thuật cao và tiêu tốn nhiều thời gian vận hành hệ thống.

Tại Việt Nam, Haravan là một nền tảng phổ biến, thân thiện với người dùng trong nước và tích hợp tốt các dịch vụ vận chuyển, thanh toán nội địa [5]. Tuy nhiên, giải pháp này vẫn có những hạn chế nhất định trong việc tùy biến giao diện nếu người dùng không biết lập trình, đồng thời hệ thống chưa được xây dựng dựa trên kiến trúc đa thuê bao thực sự tối ưu cho việc cá nhân hóa giao diện động. Đối với các doanh nghiệp lớn, Magento là một lựa chọn mạnh mẽ, nhưng đi kèm với sự phức tạp và chi phí triển khai cực kỳ đắt đỏ [6].

Tiêu chí	Shopify	WooCommerce	Haravan	ShopVolo v2
Chi phí thấp	×	✓	Trung bình	✓
Không cần lập trình	✓	×	✓	✓
Tùy biến giao diện sâu	Hạn chế	✓ (cần dev)	×	✓
Multi-tenant thực sự	✓	×	×	✓
Phù hợp SME Việt Nam	×	Khó	✓	✓
Vận hành 20k+ shop / hạ tầng nhỏ	N/A	×	N/A	✓

Bảng 1.1: So sánh các nền tảng thương mại điện tử

Bảng 1.1 tổng hợp sự khác biệt giữa các nền tảng thương mại điện tử hiện nay. Có thể thấy, các nền tảng hiện có chưa giải quyết được đồng thời ba vấn đề cốt lõi: (i) chi phí triển khai và duy trì thấp, (ii) khả năng tùy biến giao diện sâu rộng mà không cần lập trình, và (iii) khả năng vận hành một số lượng lớn cửa hàng trên một hạ tầng phần cứng tối ưu.

Đề tài hướng đến xây dựng nền tảng ShopVolo v2 nhằm giải quyết các bài toán trên với các chức năng chính bao gồm: (i) khởi tạo cửa hàng trực tuyến trong vài phút thông qua các bước hướng dẫn tự động, (ii) cung cấp công cụ tùy biến giao diện trực quan không cần lập trình, (iii) quản lý vòng đời sản phẩm và trạng thái đơn hàng đầy đủ, (iv) xử lý thanh toán trực tuyến qua các cổng thanh toán hoặc nhận hàng trả tiền, và (v) hỗ trợ ánh xạ tên miền tùy chỉnh riêng biệt cho từng cửa hàng.

1.3 Định hướng giải pháp

Để giải quyết bài toán vận hành số lượng lớn cửa hàng với chi phí hạ tầng tối ưu, đề tài lựa chọn kiến trúc phần mềm Multi-tenant SaaS kết hợp với nguyên lý thiết kế "Zero-file". Đây là phương pháp quản lý giao diện trong đó toàn bộ cấu trúc trang của các cửa hàng được biểu diễn hoàn toàn dưới dạng tệp dữ liệu cấu hình JSON thay vì các tệp mã nguồn riêng lẻ. Cách tiếp cận này cho phép một mã nguồn duy nhất có thể phục vụ hàng ngàn cửa hàng khác nhau mà không cần thực hiện quá trình triển khai lại từng ứng dụng riêng biệt.

Hệ thống ShopVolo v2 được xây dựng dưới dạng Monorepo sử dụng Turborepo, bao gồm bốn ứng dụng chính. Ứng dụng Admin Dashboard phát triển bằng Next.js 15 cho phép người bán quản lý cửa hàng toàn diện. Ứng dụng API Core viết bằng NestJS 11 cung cấp các giao diện lập trình ứng dụng REST API có khả năng cô lập dữ liệu tự động qua AsyncLocalStorage. Ứng dụng Storefront Engine cũng dùng Next.js 15 để kết xuất giao diện động từ cấu hình, cùng với một công cụ CLI Tool hỗ trợ vận hành hệ thống hàng loạt. Cơ sở dữ liệu PostgreSQL được dùng cho các dữ liệu có cấu trúc chặt chẽ như đơn hàng, trong khi MongoDB lưu trữ các dữ liệu có tính chất linh hoạt như cấu hình giao diện.

Đóng góp nổi bật của đề tài là thuật toán Deep Merge, kết hợp bộ khung giao diện Master Template mặc định với cấu hình riêng Tenant Config của từng cửa hàng. Cơ chế này chỉ lưu trữ những phần thay đổi, giúp tiết kiệm đáng kể không gian lưu trữ và kết hợp với chiến lược bộ nhớ đệm Redis có vòng đời một giờ nhằm đạt hiệu suất cao. Mục tiêu của hệ thống là có khả năng phục vụ đồng thời 20.000 cửa hàng trên một máy chủ 16GB RAM với thời gian phản hồi dưới 500ms đối với dữ liệu mới và dưới 10ms đối với dữ liệu đã lưu trong bộ nhớ đệm. Chi tiết về các giải pháp kỹ thuật này sẽ được trình bày cụ thể trong Chương 5.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày quá trình khảo sát hiện trạng từ ba nguồn chính bao gồm người dùng, hệ thống hiện có, và ứng dụng tương tự, qua đó xác định yêu cầu chức

năng thông qua mười trường hợp sử dụng, đặc tả chi tiết năm trường hợp sử dụng quan trọng nhất, và đưa ra các yêu cầu phi chức năng về hiệu năng, bảo mật, cũng như hạ tầng hệ thống.

Chương 3 giới thiệu các công nghệ và nền tảng kiến trúc được lựa chọn để hiện thực hóa các yêu cầu đã xác định ở Chương 2, bao gồm Turborepo, NestJS, Next.js, Prisma, Mongoose, Redis, MinIO, và Docker, cùng lý do chọn mỗi công nghệ thay vì các giải pháp thay thế có sẵn trên thị trường.

Chương 4 trình bày thiết kế kiến trúc tổng quan theo mô hình phân tầng, thiết kế giao diện cho trang quản trị và trang hiển thị cửa hàng, thiết kế lớp cho các thành phần chính, lược đồ cơ sở dữ liệu PostgreSQL và MongoDB, cùng với kết quả xây dựng ứng dụng, quy trình kiểm thử và mô hình triển khai hệ thống.

Chương 5 phân tích chuyên sâu hai đóng góp kỹ thuật nổi bật của đề tài là cơ chế giao diện "Zero-file" với bộ phân giải thành phần động và thuật toán Deep Merge trộn dữ liệu giữa Master Template và cấu hình tùy chỉnh, kèm theo các kết quả đo lường và đánh giá hiệu năng thực tế.

Chương 6 tổng kết các kết quả đã đạt được trong quá trình nghiên cứu và phát triển, so sánh sản phẩm hoàn thiện với các nền tảng tương tự trên thị trường, đánh giá một cách khách quan những điểm chưa hoàn thiện và đề xuất các hướng phát triển tiếp theo.

Chương này đã làm rõ bài toán thực tiễn về sự thiếu hụt một nền tảng phù hợp cho các doanh nghiệp vừa và nhỏ, đồng thời phân tích những điểm còn hạn chế của các hệ thống thương mại điện tử hiện tại. Dựa trên cơ sở đó, đề tài đề xuất việc phát triển hệ thống ShopVolo v2 ứng dụng kiến trúc đa thuê bao và nguyên lý giao diện "Zero-file" để tối ưu hóa quá trình vận hành và tùy biến. Chương tiếp theo sẽ trình bày chi tiết kết quả khảo sát hiện trạng và phân tích yêu cầu của hệ thống.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã xác định bài toán về rào cản chi phí và kỹ thuật mà các doanh nghiệp vừa và nhỏ gặp phải khi tiếp cận thương mại điện tử, đồng thời đề xuất giải pháp nền tảng ShopVolo v2. Chương 2 này sẽ đi sâu vào việc khảo sát và phân tích yêu cầu của hệ thống để làm cơ sở cho quá trình thiết kế. Cụ thể, phần 2.1 trình bày kết quả khảo sát hiện trạng từ ba nguồn chính nhằm nhận diện rõ vấn đề. Tiếp đó, phần 2.2 cung cấp cái nhìn tổng quan về các chức năng của hệ thống thông qua các biểu đồ phân rã. Phần 2.3 đi vào đặc tả chi tiết luồng nghiệp vụ của các trường hợp sử dụng cốt lõi. Cuối cùng, phần 2.4 đưa ra các yêu cầu phi chức năng cần đáp ứng để hệ thống vận hành trơn tru.

2.1 Khảo sát hiện trạng

Đối tượng người dùng của ShopVolo v2 là các doanh nghiệp vừa và nhỏ tại Việt Nam có nhu cầu mở rộng kênh bán hàng trực tuyến. Qua phân tích báo cáo về thương mại điện tử Việt Nam [7], hơn 60% các đơn vị doanh nghiệp vừa và nhỏ phản ánh chi phí xây dựng website đang là rào cản lớn nhất của họ. Người dùng đang rất cần một nền tảng thương mại điện tử thực sự dễ dùng mà không yêu cầu kiến thức viết mã, có mức chi phí thấp để duy trì, nhưng vẫn cung cấp được một giao diện chuyên nghiệp. Dựa vào những nhu cầu cấp thiết đó, hệ thống phân tích và chỉ ra ba nhóm người dùng chính bao gồm: (i) Super Admin giữ vai trò quản trị toàn bộ hệ thống nền tảng, (ii) Tenant Owner là những người kinh doanh trực tiếp quản lý cửa hàng của mình, và (iii) End Customer đại diện cho lượng khách hàng cuối cùng sẽ truy cập vào nền tảng để mua sắm.

Trước khi nền tảng ShopVolo v2 được định hình, các cửa hàng thường tự xây dựng trang web bằng các công cụ như WordPress kết hợp WooCommerce hoặc dựa hoàn toàn vào các trang mạng xã hội phổ biến như Facebook và Zalo. Tuy nhiên, các giải pháp này bộc lộ rất nhiều hạn chế cốt lõi như không có hệ thống quản lý đơn hàng tập trung, trải nghiệm giao diện không nhất quán trên các thiết bị, và gần như không thể mở rộng quy mô khi số lượng sản phẩm cũng như lượng truy cập tăng vọt.

Khi xét đến các ứng dụng phần mềm dịch vụ chuyên biệt tương tự hiện nay, Shopify nổi bật với tư cách là một nền tảng thương mại điện tử khổng lồ cung cấp giao diện bắt mắt. Điểm yếu của nền tảng này là chi phí duy trì cố định hằng tháng khá cao và khả năng tùy biến sâu bị hạn chế [3]. Trong khi đó, WooCommerce lại cung cấp tính tự do tuyệt đối nhờ mã nguồn mở nhưng người sử dụng bị ép buộc phải tự cấu hình máy chủ, vá lỗi bảo mật liên tục, gây hao tổn nguồn lực kỹ thuật

đáng kể [4].

Tại thị trường trong nước, Haravan đã xây dựng được một hệ sinh thái hỗ trợ thanh toán nội địa vô cùng tiện lợi cho các nhà bán lẻ [5]. Tuy nhiên, công cụ thiết kế của họ vẫn còn cứng nhắc, và bản thân hệ thống chưa ứng dụng triệt để kiến trúc đa thuê bao nhằm tiết kiệm tài nguyên kết xuất giao diện. Ngoài ra, phân khúc doanh nghiệp quy mô cực lớn thường chọn giải pháp Adobe Commerce với sự mạnh mẽ tuyệt đối nhưng lại đòi hỏi nguồn kinh phí khổng lồ để bắt đầu và duy trì bảo trì hệ thống [6].

Tiêu chí so sánh	Shopify	WooCommerce	Haravan	ShopVolo v2 (đề xuất)
Mô hình triển khai	SaaS	Self-hosted	SaaS	SaaS
Chi phí hàng tháng	\$29–\$299	Miễn phí (hosting riêng)	200k–3tr VND	Tối ưu (multi-tenant)
Tùy biến giao diện	Theme cố định	Cần dev WordPress	Hạn chế	Visual builder, Zero-file
Multi-tenancy thực sự	Có	Không	Không	Có (AsyncLocalStorage)
Hỗ trợ thanh toán VN	Hạn chế	Plugin	Có	Có (Stripe, COD, MoMo)
Vận hành 20k+ shop / RAM thấp	Không áp dụng	Không	Không áp dụng	Mục tiêu thiết kế

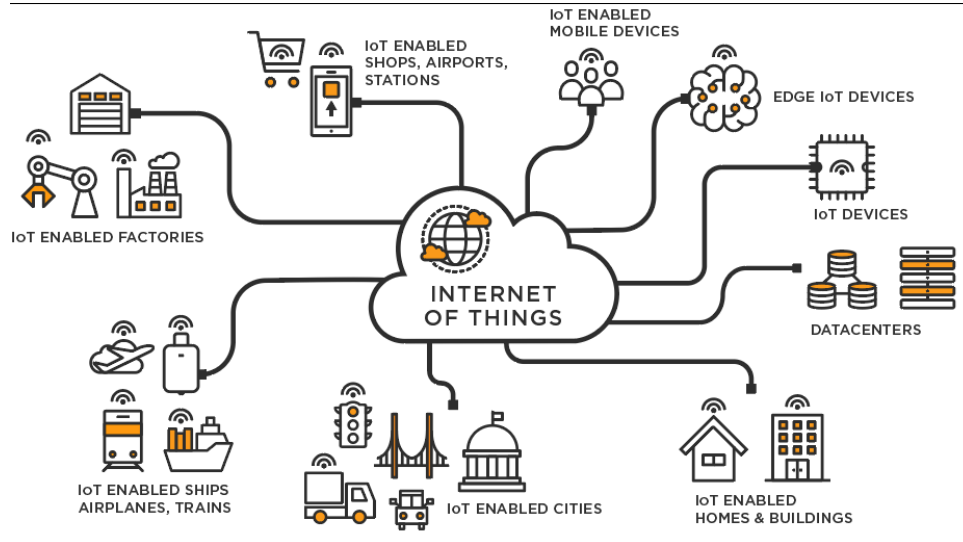
Bảng 2.1: So sánh chi tiết các nền tảng thương mại điện tử

Bảng 2.1 cho thấy rõ ràng một sự thiếu hụt nghiêm trọng trên thị trường. Các hệ thống hiện tại đều chưa thể giải quyết triệt để vấn đề cân bằng giữa chi phí, khả năng vận hành nhiều cửa hàng trên cấu hình máy chủ hạn hẹp và công cụ thiết kế giao diện linh hoạt. Từ những hạn chế tổng quát kể trên, nền tảng của đề tài phải tập trung xây dựng 10 tính năng cốt lõi: (i) khởi tạo cửa hàng, (ii) quản lý bộ khung nền tảng, (iii) tùy biến thương hiệu, (iv) quản lý sản phẩm, (v) thiết lập menu điều hướng, (vi) tạo trang nội dung động, (vii) duyệt sản phẩm động trên trang cửa hàng, (viii) quản lý giỏ hàng, (ix) thanh toán và xử lý đơn hàng, và cuối cùng là (x) ánh xạ tên miền tùy chỉnh.

2.2 Tổng quan chức năng

Phần này có nhiệm vụ tóm tắt cấu trúc phân mảnh hệ thống thông qua các sơ đồ trường hợp sử dụng từ mức tổng thể đến mức chi tiết. Việc đặc tả chuyên sâu từng chức năng cốt lõi sẽ được tiến hành ở mục tiếp theo.

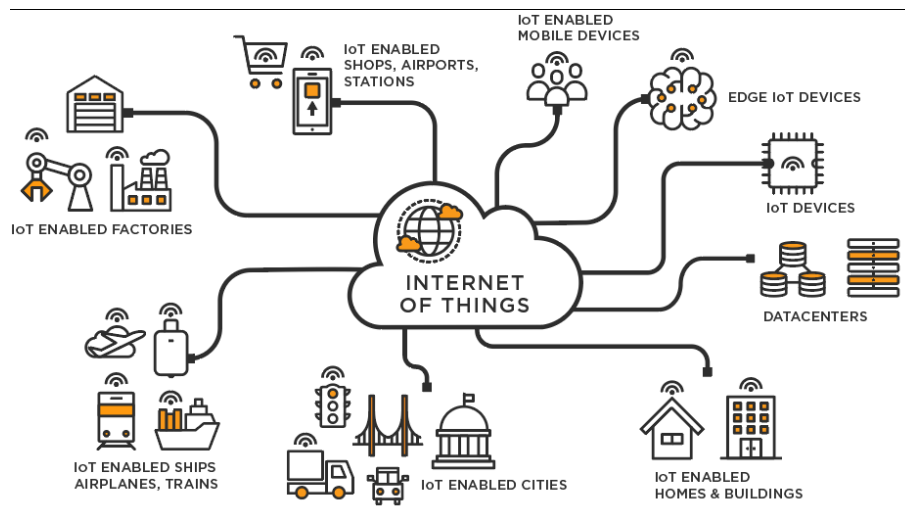
2.2.1 Biểu đồ use case tổng quát



Hình 2.1: Biểu đồ use case tổng quát hệ thống ShopVolo v2

Hình 2.1 mô tả cái nhìn toàn cảnh về mọi hành vi tương tác trong hệ thống. Tác nhân Super Admin là người nắm giữ quyền hành lớn nhất, chịu trách nhiệm quản lý toàn bộ nền tảng và thiết kế các mẫu bộ khung nền tảng cho cửa hàng. Tác nhân Tenant Owner là những người kinh doanh tham gia trực tiếp vào việc đăng ký, thiết lập, cấu hình và quản lý mọi góc ngách thông tin trong cửa hàng của riêng họ. Cuối cùng, tác nhân End Customer là tập hợp lượng khách hàng đồ sộ liên tục truy cập vào hệ thống mặt tiền hiển thị nhằm tìm kiếm sản phẩm và thanh toán.

2.2.2 Biểu đồ use case phân rã — Quản lý Shop

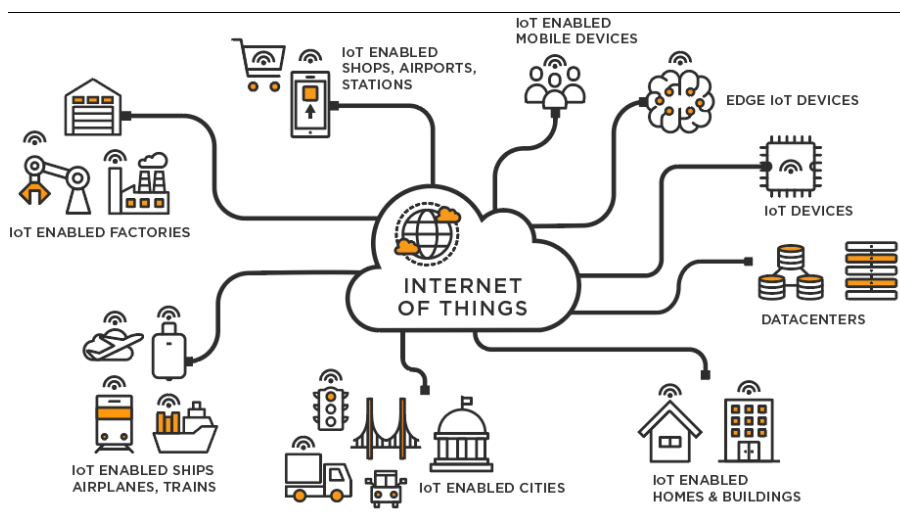


Hình 2.2: Biểu đồ use case phân rã nhóm Quản lý Shop

Hình 2.2 tập trung mô tả chi tiết các tác vụ liên quan đến việc thiết lập cửa hàng. Chức năng khởi tạo shop hỗ trợ đăng ký nhanh tài khoản và định danh cửa hàng.

Quản lý nền tảng mẫu cung cấp luồng tùy chỉnh giao diện gốc. Chức năng tùy biến thương hiệu giúp đổi mới hình ảnh mà không cần biết viết mã, còn thiết lập điều hướng và ánh xạ tên miền hỗ trợ tối đa cho việc định hình cấu trúc thông tin hoàn chỉnh.

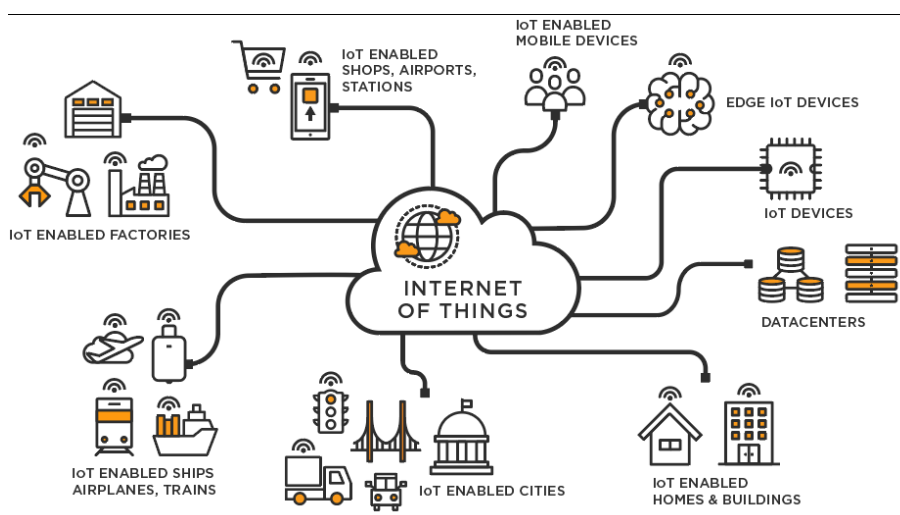
2.2.3 Biểu đồ use case phân rã — Quản lý Sản phẩm & Đơn hàng



Hình 2.3: Biểu đồ use case phân rã nhóm Sản phẩm và Đơn hàng

Hình 2.3 mô tả trực tiếp các chức năng sinh lời cốt lõi của một hệ thống bán lẻ. Quản lý sản phẩm cho phép người bán cập nhật liên tục mọi thay đổi về kho hàng và hình ảnh. Quản lý giỏ hàng đóng vai trò điểm neo kết nối mong muốn mua sắm của khách, còn chức năng thanh toán trực tiếp giải quyết dòng tiền giao dịch để hoàn tất đơn hàng một cách nhanh chóng.

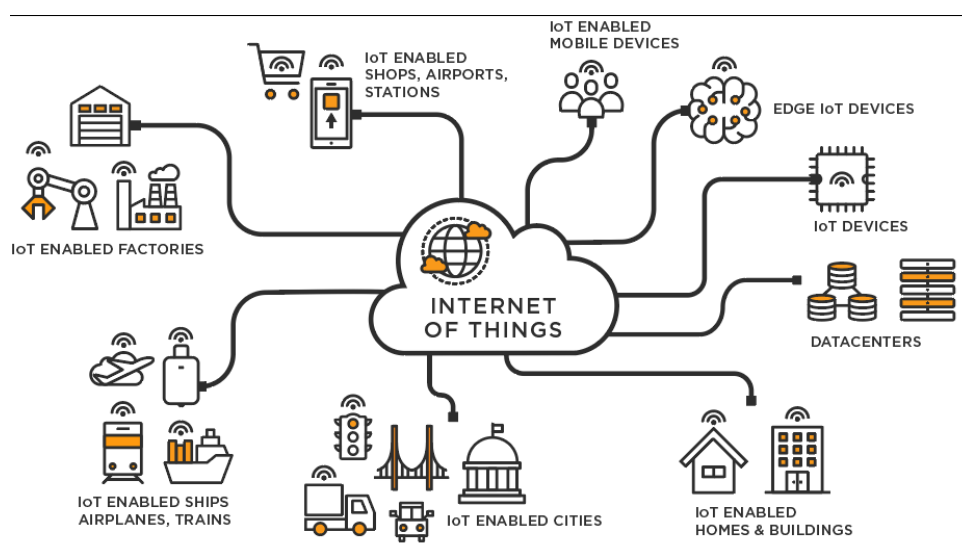
2.2.4 Biểu đồ use case phân rã — Trải nghiệm Khách hàng



Hình 2.4: Biểu đồ use case phân rã nhóm trải nghiệm Khách hàng

Hình 2.4 mô tả trải nghiệm người tiêu dùng trên bề mặt hệ thống. Khách hàng dễ dàng theo dõi thông tin cửa hàng qua các trang nội dung động, đồng thời liên tục tương tác với mặt tiền của cửa hàng thông qua chức năng duyệt sản phẩm được kết xuất mạnh mẽ từ cấu hình hệ thống.

2.2.5 Quy trình nghiệp vụ — Onboarding và Dynamic Rendering



Hình 2.5: Biểu đồ hoạt động quy trình Onboarding và Rendering động

Hình 2.5 mô tả quy trình nghiệp vụ quan trọng nhất, gắn kết vòng đời của cửa hàng từ khâu kiến tạo đến khâu hoạt động thực tế. Khi người dùng mới thực hiện đăng ký thông tin, hệ thống tự động sinh tài khoản, gán bộ khung giao diện tiêu chuẩn và cho phép cấu hình ngay lập tức. Cấu hình được xuất bản và mỗi khi có khách hàng tiến vào thông qua mạng lưới tên miền, máy chủ kết xuất lập tức kéo tệp dữ liệu về để tiến hành phép kết hợp đệ quy sâu. Toàn bộ các thông tin trộn lẫn này được hiển thị ngay lập tức lên màn hình một cách liền mạch nhất.

2.3 Đặc tả chức năng

Dựa trên mười tác vụ chính đã thống kê, phân hệ hệ thống này lựa chọn tiến hành đặc tả chuyên sâu năm quy trình vận hành trung tâm ảnh hưởng trực tiếp đến hiệu quả hoạt động tổng thể.

2.3.1 Đặc tả use case: Khởi tạo Shop (UC-01)

Tên use case	Khởi tạo Shop
Tác nhân	Tenant Owner
Mục đích	Đăng ký tài khoản
Tiền điều kiện	Tenant Owner tồn tại trong hệ thống
Hậu điều kiện	Shop được tạo thành công và email xác nhận đã được gửi
Luồng chính	1. Tenant Owner nhập shopName, email và password 2. Hệ thống kiểm tra email và domain chưa trùng lặp 3. Hệ thống tạo bản ghi tenant trong PostgreSQL 4. Hệ thống tạo user với role=OWNER, liên kết với tenant 5. Hệ thống gán Master Template mặc định vào MongoDB (TenantLayout) 6. Hệ thống khởi tạo navigation menu mặc định 7. Hệ thống gửi email xác nhận 8. API trả về tenantId, domain, status=active
Luồng phát sinh	[2a] Domain đã tồn tại → HTTP 409, code 9996, message "Domain already registered" [2b] Email đã đăng ký → HTTP 409, code 9996, message "Email already registered" [2c] Dữ liệu không hợp lệ → HTTP 400, code 1002/1003/1004

Bảng 2.2: Đặc tả use case Khởi tạo Shop (UC-01)

Bảng 2.2 mô tả luồng đăng ký ban đầu thông qua một cấu trúc hàm tự động giúp thiết lập cơ sở dữ liệu phân tán ngay tức thì cho một cửa hàng mới.

2.3.2 Đặc tả use case: Tùy biến giao diện (UC-03)

Tên use case	Tùy biến
Tác nhân	Tenant C
Mục đích	Thay đổi hàng
Tiền điều kiện	Tenant C active
Hậu điều kiện	Cấu hình (TenantL giao diện
Luồng chính 2. Dashboard gửi PATCH /api/v1/tenants/{tenantId}/config với branding JSON 3. API nhận override, thực hiện deep merge với Master Template 4. Lưu chỉ phần override (diff) vào MongoDB TenantLayout 5. Invalidate Redis cache key storefront:config:{tenantId} 6. Trả về config đã cập nhật và lastModified	1. Tenant Admin D
Luồng phát sinh [3b] Màu sắc không đúng hex format → HTTP 400, message "Invalid color format"	[3a] UR "Invalid

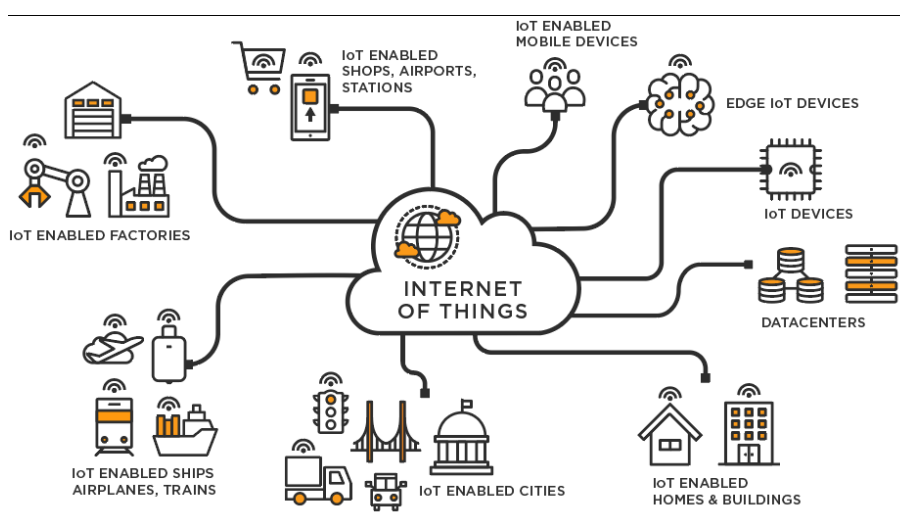
Bảng 2.3: Đặc tả use case Tùy biến thương hiệu (UC-03)

Bảng 2.3 mô tả quy trình tùy biến giúp lưu trữ lại đúng phần nội dung khác biệt thay vì toàn bộ mã nguồn, tối ưu hóa hệ thống dữ liệu đồ sộ.

2.3.3 Đặc tả use case: Quản lý sản phẩm (UC-04)

Tên use case
Tác nhân
Mục đích
Tiền điều kiện
Hậu điều kiện
Luồng chính
2. Admin Dashboard gửi yêu cầu cập nhật API hệ thống lỗi 3. Hình ảnh thô đẩy trực tiếp lên kho chứa đối tượng MinIO 4. Trí tuệ nhân tạo nhận diện và bóc tách phong nền chuẩn bị qua CronJob máy chủ ảnh 5. Hình ảnh sau bóc tách được thu nhỏ theo ba mức độ phân giải tiêu chuẩn 6. Toàn bộ định danh ảnh thu nhỏ lưu ngược vào bộ nhớ MongoDB 7. Thông báo hoàn tất khởi tạo sản phẩm về màn hình người dùng
Luồng phát sinh
[4a] Lỗi kỹ thuật máy chủ ảnh ngắt quãng → Hệ thống lưu cấu hình ảnh mặc định và kích hoạt lại sa

Bảng 2.4: Đặc tả use case Quản lý sản phẩm (UC-04)



Hình 2.6: Biểu đồ hoạt động luồng tải lên và xử lý ảnh sản phẩm

Hình 2.6 là quy trình quản lý thông tin sản phẩm và phân luồng làm sạch phong nền ảnh thông minh giúp tăng độ sắc nét sản phẩm một cách chuyên nghiệp.

2.3.4 Đặc tả use case: Duyệt sản phẩm động (UC-07)

Tên use case
Tác nhân
Mục đích
Tiền điều kiện
Hậu điều kiện
Luồng chính 2. Lớp phần mềm trung gian tự động bóc tách chuỗi để xác định mã cửa hàng 3. Động cơ mặt tiền kích hoạt hàm GET /api/v1/storefront/config?domain=xxx 4. Máy chủ ứng dụng lập tức lôi cấu hình đệ quy sâu từ kho lưu trữ nền 5. Bản ghi nội dung trộn lẫn được đẩy thẳng vào vùng nhớ tạm Redis với tuổi thọ giới hạn một 6. Bộ phân giải thành phần cấu trúc JSON sang bộ giao diện lập trình bề mặt React 7. Nội dung phản hồi được truyền tới màn hình phía khách truy cập hoàn hảo nhất
Luồng phát sinh [4a] Lỗi đường truyền tới MongoDB ngắt quãng → Hệ thống hiển thị cảnh báo bảo trì máy chủ

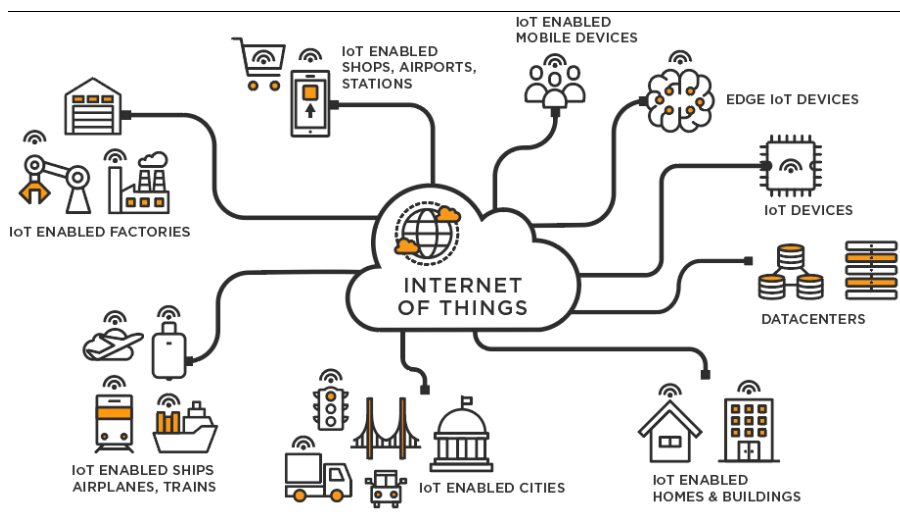
Bảng 2.5: Đặc tả use case Duyệt sản phẩm động (UC-07)

Bảng 2.5 tập trung biểu diễn quy trình xử lý luồng sự kiện cốt lõi định hình kiến trúc Zero-file khi hệ thống phản ứng linh động thông qua các tập tin siêu văn bản JSON theo thời gian thực.

2.3.5 Đặc tả use case: Thanh toán và Đơn hàng (UC-09)

Tên use case
Tác nhân
Mục đích
Tiền điều kiện
Hậu điều kiện
Luồng chính
2. Mặt tiền gọi hàm POST /api/v1/orders lên hệ thống trung tâm quản trị cơ sở dữ liệu 3. Hệ thống tạo nháp đơn hàng cùng mức thanh toán tạm thời chờ kết quả 4. Quá trình kết nối cổng thanh toán kích hoạt, mã giao dịch được lưu vết lại 5. Cổng Gateway gửi hàm callback webhook xác nhận tiền đã chảy vào tài khoản 6. Hệ thống lỗi đối soát verify signature của webhook, xác thực an toàn tuyệt đối 7. Lệnh cập nhật trạng thái update order status được tiến hành, kéo theo gửi email xác nhận và bảng
Luồng phát sinh
[6a] Mã chứng thực chữ ký webhook không khớp chuẩn → Kích hoạt báo động xâm nhập và từ chối

Bảng 2.6: Đặc tả use case Thanh toán và Đơn hàng (UC-09)



Hình 2.7: Biểu đồ hoạt động quy trình thanh toán và xác nhận đơn hàng

Hình 2.7 xác nhận chặt chẽ cơ chế đóng kín dữ liệu thanh toán với bên thứ ba một cách an toàn.

2.4 Yêu cầu phi chức năng

Hệ thống cần đáp ứng thời gian phản hồi dưới 500ms cho cold cache và dưới 10ms cho warm cache của hệ thống Redis. Mục tiêu vận hành đồng thời 20.000

shop trên hạ tầng cấu hình khiêm tốn 16GB RAM đặt ra yêu cầu vô cùng nghiêm ngặt về tối ưu bộ nhớ. Cụ thể, NestJS phải bị giới hạn cỡ cấu trúc cấp phát vùng nhớ tối đa 1024MB, cả hai cụm cơ sở dữ liệu PostgreSQL và MongoDB phân bổ mỗi thành phần tối đa 512MB RAM, còn đối với máy chủ truy xuất đệm Redis được thiết lập vòng lặp xử lý không vượt quá 256MB. Xét về độ tin cậy, hệ thống cam kết duy trì chuẩn hoạt động ở mức uptime đạt từ 99.5% trở lên, đồng thời không cộng dồn thời gian đóng cổng để cấu hình bảo trì. Một cơ chế quy trình tự động được thiết kế nhằm sao lưu chạy mỗi ngày thông qua các tiến trình Kubernetes CronJob khép kín. Mọi lỗi bất ngờ phát sinh từ cơ sở dữ liệu đều được quản lý chặt chẽ qua cơ chế ném ngoại lệ `InternalServerErrorException` của NestJS, tuyệt đối không để lộ mã ngăn xếp truy vết ra thông báo trả về cho máy khách. Đi sâu hơn về khía cạnh tính dễ dùng, toàn bộ trung tâm điều khiển Admin Dashboard cần hỗ trợ tối ưu cho các người dùng cuối không có kiến thức kỹ thuật nâng cao. Giao diện thiết kế phải tuân thủ nghiêm chỉnh những quy định thiết kế tối giản, và mọi tính năng tác vụ trọng yếu như đăng hình ảnh gốc, thay màu nền thương hiệu phải dễ dàng vận hành với tổng thời gian thao tác tối đa không vượt qua thời lượng 3 phút. Phân tích đến yếu tố bảo mật, quá trình cấp quyền chứng thực xác thực thông qua cấu trúc JWT chuẩn mã hóa HS256. Mọi đường dẫn truy cập API đều yêu cầu khắt khe thông số xác minh Authorization header ngay ở những truy vấn đơn giản nhất. Đặc biệt, bản chất môi trường thiết lập đa khách thuê đặt nền móng chắc chắn đảm bảo cô lập dữ liệu tuyệt đối; chủ cửa hàng này sẽ hoàn toàn bị chặn đứng quyền truy xuất dữ liệu từ bất cứ cửa hàng nào khác trong cùng máy chủ. Secret mã hóa cực đoạn được nạp qua cấu hình biến môi trường và không bao giờ xuất hiện lộ liễu trên các mã nguồn vật lý, đồng thời mã hóa TLS 1.3 bảo trợ toàn vẹn luồng dữ liệu liên thông. Đứng ở các góc độ yêu cầu kỹ thuật nền tảng, hệ thống cấu hình chạy tốt trên máy chủ có nhân lõi Node.js phiên bản 22 trở lên. Hệ thống phía máy khách hỗ trợ ổn định cho trình duyệt Chrome phiên bản 100, Firefox bản 100 hay Safari từ bản 15. Dữ liệu chạy dựa trên quy chuẩn cấu trúc PostgreSQL 16 tích hợp ORM Prisma và MongoDB 7 đi kèm Mongoose linh động. Hạ tầng triển khai kết dính toàn cục thông qua Docker Compose trong khi vận hành phát triển và nâng cấp thành hệ quản trị Kubernetes phân luồng tại máy chủ sản xuất, tất cả đồng loạt được thiết kế và quản trị theo cấu trúc tự động Turborepo nhằm gia tăng sức bật thời gian đóng gói biên dịch hệ thống.

Chương này đã làm rõ hiện trạng thị trường khi khảo sát ba nguồn phân tích cụ thể, từ đó hệ thống đã định hướng và chỉ điểm rõ 10 nhóm tính năng cốt lõi. Từ 10 chức năng quan trọng này, chương cũng đã đặc tả cẩn thận từng sự kiện diễn ra của 5 trường hợp sử dụng ưu tú nhất, bao gồm luôn những yêu cầu phi chức năng cần

đáp ứng ở bước chạy đà thực tế. Chương tiếp theo trình bày các công nghệ được lựa chọn để hiện thực hóa các yêu cầu đã xác định.

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

Chương 2 đã xác định các yêu cầu chức năng và phi chức năng của hệ thống ShopVolo v2 từ quá trình khảo sát kỹ lưỡng thực tiễn người dùng. Chương 3 này sẽ trình bày các công nghệ được lựa chọn để đáp ứng những yêu cầu đó, đồng thời phân tích rõ lý do tại sao chúng được tin dùng hơn so với các giải pháp thay thế trên thị trường. Về mặt tổng thể, nội dung sẽ được liệt kê sơ bộ thông qua các nhóm công nghệ lõi như kiến trúc quản lý tập trung mã nguồn, hệ thống lập trình máy chủ phụ trợ, nền tảng phát triển giao diện hiển thị, cấu trúc cơ sở dữ liệu chuyên biệt, và cuối cùng là các công nghệ đóng gói hạ tầng cũng như lưu trữ đối tượng.

3.1 Turborepo — Quản lý Monorepo

Để đáp ứng yêu cầu phi chức năng tại 2.4 về tính bảo trì và quy trình xây dựng tự động hóa, hệ thống yêu cầu quản lý 4 ứng dụng (admin, api-core, storefront, cli-tool) và 5 gói phần mềm dùng chung (ui-library, schema, database, i18n, master-templates) một cách nhất quán trong cùng một kho lưu trữ tập trung.

Các công cụ quản lý kho chứa mã nguồn tập trung phổ biến hiện nay bao gồm (i) Nx, (ii) Lerna, và (iii) trình quản lý không gian làm việc pnpm thuần túy.

Turborepo được chọn vì (i) khả năng lưu trữ bộ nhớ đệm dạng tăng dần giúp hệ thống chỉ tiến hành biên dịch lại đúng các gói phần mềm thực sự bị thay đổi, giúp giảm triệt để thời gian chạy các luồng tự động, (ii) công cụ này thiết lập được quy trình biên dịch tự động dựa vào khái niệm đồ thị có hướng phi chu trình, và (iii) không yêu cầu quá trình chuyển đổi cấu trúc phức tạp khi tích hợp sẵn với không gian làm việc của npm đã tồn tại [8].

3.2 NestJS — Framework Backend

Để đáp ứng các chức năng được thiết kế chi tiết tại 2.3, hệ thống yêu cầu một giải pháp xây dựng giao diện lập trình ứng dụng có cấu trúc mô-đun rõ ràng như quản lý cửa hàng, sản phẩm, đơn hàng, đồng thời hỗ trợ tiêm phụ thuộc, xây dựng tầng trung gian phục vụ đa thuê bao và hỗ trợ định dạng tĩnh cấu trúc mạnh mẽ.

Các công cụ lập trình phụ trợ bằng ngôn ngữ JavaScript nổi bật khác bao gồm (i) Express.js, (ii) Fastify, và (iii) Hono.

NestJS được chọn vì (i) kiến trúc mô-đun mô phỏng hệ thống Angular hỗ trợ tổ chức mã nguồn theo từng cụm giới hạn rõ ràng, (ii) tích hợp rất tốt với bộ nhớ ngữ cảnh bất đồng bộ nhằm truyền tải định danh khách thuê một cách tự động, và (iii) sử dụng trình trang trí kết hợp với cơ chế lọc ngoại lệ giúp xử lý thống nhất các lỗi nội bộ trên toàn bộ máy chủ [9].

Cụ thể, tầng trung gian tự động giải mã thông tin định danh khách thuê từ phần mào đầu truy vấn, sau đó lưu vào bộ nhớ bất đồng bộ để toàn bộ quá trình đọc ghi cơ sở dữ liệu tiếp theo đều tự động gắn kết.

3.3 Next.js — Framework Frontend

Để đáp ứng yêu cầu hiệu năng tại 2.4, hệ thống đòi hỏi nền tảng mặt tiền phải phản hồi ở tốc độ dưới mức 500 mili-giây, cùng với đó trang quản trị của người bán phải tải được các dạng biểu mẫu cấu trúc phức tạp với độ tương tác phản ứng tức thì.

Các nền tảng xây dựng giao diện nổi bật thay thế bao gồm (i) bộ đôi React và Vite, (ii) Remix, và (iii) hệ thống Nuxt.js dựa trên Vue.

Next.js phiên bản 15 được chọn vì (i) bộ định tuyến ứng dụng hỗ trợ tính năng kết xuất các phần tử phía máy chủ giúp giảm thiểu dung lượng mã tải về cho khách truy cập, (ii) các hàm trung gian hoạt động ở biên cho phép nhận diện tên miền khách thuê và định tuyến từ rất sớm, và (iii) cơ chế tái tạo tĩnh tăng dần cung cấp khả năng tự động thiết lập thời gian sống cho bộ nhớ đệm của các trang mô tả sản phẩm một cách cực kỳ linh hoạt [10].

3.4 PostgreSQL và Prisma ORM — Cơ sở dữ liệu quan hệ

Để đáp ứng việc quản lý các thực thể có cấu trúc định hình cố định đã được xác định tại 2.1 như chủ cửa hàng, tài khoản, đơn vị mua hàng, hệ thống đòi hỏi một giải pháp bảo đảm tính nguyên tử của giao dịch, ràng buộc chặt chẽ khóa ngoại và khả năng trích xuất thông tin bảng thống kê nâng cao.

Các giải pháp quản lý bảng biểu cấu trúc thay thế bao gồm (i) MySQL 8, (ii) SQLite ở môi trường cục bộ, và (iii) hệ thống phân tán CockroachDB.

PostgreSQL 16 kết hợp công cụ Prisma được chọn vì (i) có khả năng hỗ trợ kiểu định dạng JSONB nhằm lưu vết các tham số động cực tốt khi cần, (ii) cung cấp chức năng bảo mật truy xuất cấp độ hàng dữ liệu hỗ trợ mạnh mẽ cho định hướng thiết kế đa thuê bao, và (iii) công cụ Prisma ORM tự động khởi tạo hệ sinh thái kiểu tĩnh và phiên bản hóa các đợt di chuyển cấu trúc mạnh mẽ [11], [12].

3.5 MongoDB và Mongoose — Cơ sở dữ liệu tài liệu

Để đáp ứng nền tảng kiến trúc Zero-file đã vạch ra tại 1.3, việc lưu trữ các tệp cấu hình JSON phải giải quyết bài toán lồng nhau sâu sắc của bộ khung nền và bộ tùy chỉnh cửa hàng, đồng thời hỗ trợ cập nhật mở rộng không cần theo định dạng cố định khắt khe của kho lưu trữ quan hệ truyền thống.

Các nền tảng giải quyết dữ liệu văn bản phi cấu trúc khác gồm (i) CouchDB, (ii) giải pháp đám mây DynamoDB, và (iii) lợi dụng kiểu JSONB của chính cơ sở dữ

liệu PostgreSQL.

MongoDB phiên bản 7 kết hợp bộ thư viện Mongoose được chọn vì (i) mô hình dữ liệu tài liệu cực kỳ tự nhiên khi diễn dịch các khối phần tử giao diện lồng ghép, (ii) tính năng thẩm định dữ liệu linh động của Mongoose cho phép kiểm soát chất lượng tham số cốt lõi mà vẫn chứa lại không gian để linh hoạt biến hóa, và (iii) thuật toán kết hợp đệ quy sâu dễ dàng tích hợp để đồng bộ các mảng dữ liệu đồ sộ [13].

3.6 Redis — Cache In-memory

Để đáp ứng yêu cầu hiệu năng cực kỳ khắt khe tại 2.4, hệ thống đặt mục tiêu phản hồi trong 10 mili-giây khi duyệt trang phục vụ cho việc lấy cấu hình ở phần 2.3.4, điều hoàn toàn bất khả thi nếu cứ liên tục gọi cấu trúc dữ liệu thô.

Các phương pháp thay thế giải quyết độ trễ phổ biến bao gồm (i) Memcached, (ii) kiểm soát bộ nhớ qua giao thức HTTP cơ bản, và (iii) sử dụng mạng phân phối nội dung thuần túy.

Redis được chọn vì (i) hệ lưu trữ trên vùng nhớ trực tiếp đảm bảo độ trễ ở mức vô cùng hoàn hảo, (ii) khả năng thiết lập tuổi thọ tự động hết hạn mà không cần tốn thêm tài nguyên máy chủ để tự thu dọn rác, và (iii) giao thức phát tín hiệu độc lập rất hữu ích cho thao tác vô hiệu hóa bộ đệm khi cửa hàng thực hiện thao tác cập nhật giao diện hiển thị [14].

3.7 MinIO — Object Storage

Để đáp ứng quy trình quản lý thông tin ở phần 2.3.3, hệ thống yêu cầu một giải pháp lưu trữ tệp tin hình ảnh sản phẩm định dạng nhị phân theo nhiều biến thể kích cỡ, đây là một môi trường dữ liệu không thể dồn nén trực tiếp vào kho lưu trữ bảng biểu truyền thống.

Các dịch vụ hỗ trợ cung cấp kho chứa đối tượng thay thế thường được tiếp cận là (i) hệ sinh thái đám mây AWS S3, (ii) không gian lưu trữ Cloudflare R2, và (iii) ghi tệp thẳng vào hệ thống lưu trữ tĩnh trên máy chủ nội bộ.

MinIO được chọn vì (i) sở hữu chuẩn giao thức giao tiếp y hệt như cấu trúc S3, giúp việc di dời toàn bộ dữ liệu hệ thống lên nền tảng trực tuyến lớn không gặp trở ngại gì sau này, (ii) bản thân phần mềm cung cấp khả năng lưu trữ vận hành ngay trên hạ tầng đóng gói khép kín giúp tiết kiệm kinh phí tối đa, và (iii) khả năng ủy quyền trực tiếp liên kết giúp trang mặt tiền có thể đưa dữ liệu lên máy chủ ảnh bỏ qua khâu xử lý luân chuyển ở trung tâm [15].

3.8 Better Auth — Xác thực và Phân quyền

Để đáp ứng đầy đủ yêu cầu bảo mật ở 2.4, toàn hệ thống phụ thuộc vào cơ chế kiểm soát truy cập phân tầng cho quản trị viên và chủ cửa hàng theo định dạng thẻ thông hành JWT, kèm theo thông số định danh thiết lập đa thuê bao.

Các công cụ lập trình xác thực bên thứ ba khả dĩ cho giải pháp này bao gồm (i) thư viện chuẩn NextAuth.js, (ii) sử dụng tự động Passport.js kết hợp thẻ xác thực, và (iii) hệ thống dịch vụ Clerk.

Thư viện Better Auth được chọn vì (i) xây dựng bằng ngôn ngữ tĩnh TypeScript bảo vệ hệ thống tuyệt đối ở cấp độ biên dịch, (ii) khả năng liên kết tự nhiên ngay trên kiến trúc định tuyến nâng cao của ứng dụng mặt tiền, và (iii) hỗ trợ kết nối thẳng vào trình điều khiển Prisma để đồng bộ phiên làm việc của người dùng vào hệ thống lưu trữ truyền thống [16].

3.9 Docker và Kubernetes — Containerization và Orchestration

Để đáp ứng yêu cầu phi chức năng về độ tin cậy được thể hiện tại 2.4, tất cả hệ quản trị phụ trợ khổng lồ phải được giữ cho vận hành đồng bộ và ổn định ở mọi phiên bản triển khai, chưa kể đến yêu cầu bắt buộc phải sao lưu tự động định kỳ vào mỗi ngày cuối ngày.

Các phương pháp triển khai quản lý cụm máy chủ truyền thống có thể kể đến như (i) cấu hình trực tiếp thủ công bằng tiện ích hệ điều hành, (ii) trình quản lý Docker Swarm tập trung, và (iii) hệ thống phân phối của Nomad.

Sự kết hợp giữa Docker và Kubernetes được chọn vì trong giai đoạn phát triển, hệ thống tự chứa cấp cho tốc độ khởi chạy tối đa, còn khi vận hành sản xuất thì (i) hệ sinh thái CronJob điều phối chuẩn xác các tác vụ dọn dẹp và dự phòng theo kế hoạch, (ii) chiến lược thay thế cuộn chiếu cho phép cập nhật phiên bản diễn ra hoàn toàn không làm ngưng trệ bất cứ tương tác nào, và (iii) trình quản lý bí mật vô hiệu hóa hoàn toàn mọi rủi ro lộ lọt cấu hình riêng tư [17], [18].

Công nghệ	Phiên bản	Loại	Mục đích trong đề tài
Turborepo	2.x	Build Tool	Quản lý monorepo, incremental build cache
NestJS	11.x	Backend Framework	REST API, multi-tenancy, DI container
Next.js	15.x	Frontend Framework	Admin Dashboard + Storefront Engine (SSR)
PostgreSQL	16	RDBMS	Lưu Tenant, User, Order, Product (structured)
Prisma	5.x	ORM	Type-safe query + schema migration
MongoDB	7	Document DB	Lưu Layout, Config, Navigation (flexible JSON)
Mongoose	8.x	ODM	Schema validation cho MongoDB
Redis	7.x	In-memory Cache	Cache storefront config (TTL=1h)
MinIO	Latest	Object Storage	Lưu ảnh sản phẩm, S3-compatible
Better Auth	1.x	Auth Library	JWT, session, role-based access
Docker	26.x	Containerization	Môi trường development nhất quán
Kubernetes	1.29+	Orchestration	Production deployment, CronJob
Tailwind CSS	4.x	CSS Framework	Styling Admin Dashboard + Storefront
Zod	3.x	Validation	Schema validation API request/response

Bảng 3.1: Tổng hợp công nghệ sử dụng trong hệ thống ShopVolo v2

Chương này đã làm rõ tất cả các công nghệ được chọn để đáp ứng tương ứng từng yêu cầu cốt lõi từ Chương 2. Nhấn mạnh sự bổ sung linh hoạt và hợp lý giữa PostgreSQL trong quản trị các cấu trúc dữ liệu cứng nhắc và MongoDB trong xử lý thông tin mềm dẻo đa biến, cũng như sự đồng hành giữa Docker Compose lúc phát triển và Kubernetes ở giai đoạn vận hành thực tế. Trên cơ sở các công nghệ đã lựa chọn, Chương 4 trình bày thiết kế kiến trúc và kết quả triển khai hệ thống.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

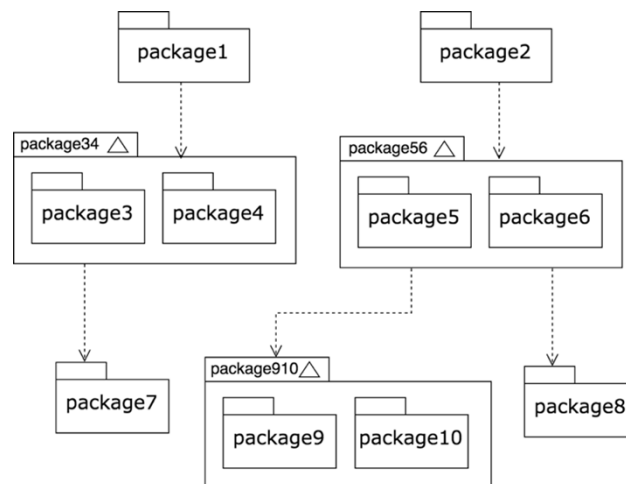
4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Mục này có độ dài từ một đến ba trang. Sinh viên cần lựa chọn kiến trúc phần mềm cho ứng dụng của mình như: kiến trúc ba lớp MVC, MVP, SOA, Microservice, v.v. rồi giải thích sơ bộ về kiến trúc đó (không giải thích chi tiết/dài dòng). Sử dụng kiến trúc phần mềm đã chọn ở trên, sinh viên mô tả kiến trúc cụ thể cho ứng dụng của mình. Gợi ý: sinh viên áp dụng lý thuyết chung vào hệ thống/sản phẩm của mình như thế nào, có thay đổi, bổ sung hoặc cải tiến gì không. Ví dụ, thành phần M trong kiến trúc lý thuyết MVC sẽ là những thành phần cụ thể nào (ví dụ: là interface I + class C1 + class C2, v.v.) trong kiến trúc phần mềm của sinh viên.

4.1.2 Thiết kế tổng quan

Sinh viên vẽ biểu đồ gói UML (UML package diagram), nêu rõ sự phụ thuộc giữa các gói (package). SV cần vẽ các gói sao cho chúng được phân theo các tầng rõ ràng, không được sắp đặt package lộn xộn trong hình vẽ. Sinh viên chú ý các quy tắc thiết kế (Các gói không phụ thuộc lẫn nhau, gói tầng dưới không phụ thuộc gói tầng trên, không phụ thuộc bỏ qua tầng, v.v.) và cần giải thích sơ lược về mục đích/nhiệm vụ của từng package. SV tham khảo ví dụ minh họa trong Hình 4.1



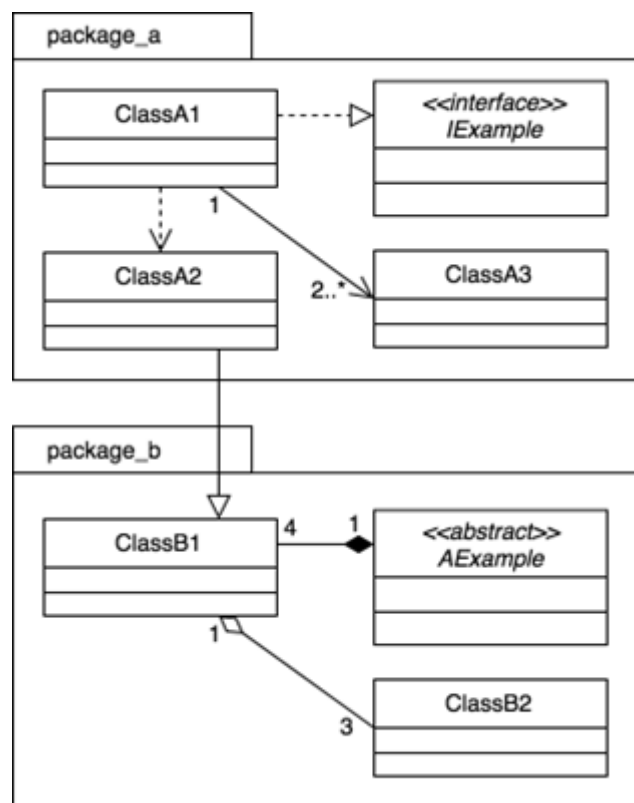
Hình 4.1: Ví dụ biểu đồ phụ thuộc gói

4.1.3 Thiết kế chi tiết gói

Sinh viên thiết kế và lần lượt vẽ biểu đồ thiết kế cho từng package, hoặc một nhóm các package liên quan để giải quyết một vấn đề gì đó. Khi vẽ thiết kế gói, sinh viên chỉ cần đưa tên lớp, không cần chỉ ra các thành viên phương thức và thuộc tính. SV tham khảo ví dụ minh họa trong Hình 4.2.

Sinh viên cần vẽ rõ ràng quan hệ giữa các lớp trong biểu đồ. Các quan hệ bao gồm: phụ thuộc (dependency), kết hợp (association), kết tập (aggregation), hợp thành (composition), kế thừa (inheritance), và thực thi (implementation). Các quan hệ này đều đã được minh họa trong 4.2.

Sau khi vẽ hình minh họa, sinh viên cần giải thích ngắn gọn về thiết kế của mình.



Hình 4.2: Ví dụ thiết kế gói

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Phần này có độ dài từ hai đến ba trang. Sinh viên đặc tả thông tin về màn hình mà ứng dụng của mình hướng tới, bao gồm độ phân giải màn hình, kích thước màn hình, số lượng màu sắc hỗ trợ, v.v. Tiếp đến, sinh viên đưa ra các thống nhất/chuẩn hóa của mình khi thiết kế giao diện như thiết kế nút, điều khiển, vị trí hiển thị thông điệp phản hồi, phối màu, v.v. Sau cùng sinh viên đưa ra một số hình ảnh minh họa thiết kế giao diện cho các chức năng quan trọng nhất. Lưu ý, sinh viên không nhầm lẫn giao diện thiết kế với giao diện của sản phẩm sau cùng.

4.2.2 Thiết kế lớp

Phần này có độ dài từ ba đến bốn trang. Sinh viên trình bày thiết kế chi tiết các thuộc tính và phương thức cho một số lớp chủ đạo/quan trọng nhất của ứng dụng

(từ 2-4 lớp). Thiết kế chi tiết cho các lớp khác, nếu muốn trình bày, sinh viên đưa vào phần phụ lục.

Để minh họa thiết kế lớp, sinh viên thiết kế luồng truyền thông điệp giữa các đối tượng tham gia cho 2 đến 3 use case quan trọng nào đó bằng biểu đồ trình tự (hoặc biểu đồ giao tiếp).

4.2.3 Thiết kế cơ sở dữ liệu

Phần này có độ dài từ hai đến bốn trang. Sinh viên thiết kế, vẽ và giải thích biểu đồ thực thể liên kết (E-R diagram). Từ đó, sinh viên thiết kế cơ sở dữ liệu tùy theo hệ quản trị cơ sở dữ liệu mà mình sử dụng (SQL, NoSQL, Firebase, v.v.)

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Sinh viên liệt kê các công cụ, ngôn ngữ lập trình, API, thư viện, IDE, công cụ kiểm thử, v.v. mà mình sử dụng để phát triển ứng dụng. Mỗi công cụ phải được chỉ rõ phiên bản sử dụng. SV nên kẻ bảng mô tả tương tự như Bảng ???. Nếu có nhiều nội dung trình bày, sinh viên cần xoay ngang bảng.

Mục đích	Công cụ	Địa chỉ URL
IDE lập trình	Eclipse Oxygen a64 bit	http://www.eclipse.org/
v.v.	v.v.	v.v.

Bảng 4.1: Danh sách thư viện và công cụ sử dụng

4.3.2 Kết quả đạt được

Sinh viên trước tiên mô tả kết quả đạt được của mình là gì, ví dụ như các sản phẩm được đóng gói là gì, bao gồm những thành phần nào, ý nghĩa, vai trò?

Sinh viên cần thống kê các thông tin về ứng dụng của mình như: số dòng code, số lớp, số gói, dung lượng toàn bộ mã nguồn, dung lượng của từng sản phẩm đóng gói, v.v. Tương tự như phần liệt kê về công cụ sử dụng, sinh viên cũng nên dùng bảng để mô tả phần thông tin thống kê này.

4.3.3 Minh họa các chức năng chính

Sinh viên lựa chọn và đưa ra màn hình cho các chức năng chính, quan trọng, và thú vị nhất. Mỗi giao diện cần phải có lời giải thích ngắn gọn. Khi giải thích, sinh viên có thể kết hợp với các chú thích ở trong hình ảnh giao diện.

4.4 Kiểm thử

Phần này có độ dài từ hai đến ba trang. Sinh viên thiết kế các trường hợp kiểm thử cho hai đến ba chức năng quan trọng nhất. Sinh viên cần chỉ rõ các kỹ thuật

kiểm thử đã sử dụng. Chi tiết các trường hợp kiểm thử khác, nếu muốn trình bày, sinh viên đưa vào phần phụ lục. Sinh viên sau cùng tổng kết về số lượng các trường hợp kiểm thử và kết quả kiểm thử. Sinh viên cần phân tích lý do nếu kết quả kiểm thử không đạt.

4.5 Triển khai

Sinh viên trình bày mô hình và/hoặc cách thức triển khai thử nghiệm/thực tế. Ứng dụng của sinh viên được triển khai trên server/thiết bị gì, cấu hình như thế nào. Kết quả triển khai thử nghiệm nếu có (số lượng người dùng, số lượng truy cập, thời gian phản hồi, phản hồi người dùng, khả năng chịu tải, các thống kê, v.v.)

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này có độ dài tối thiểu 5 trang, tối đa không giới hạn.¹ Sinh viên cần trình bày tất cả những nội dung đóng góp mà mình thấy tâm đắc nhất trong suốt quá trình làm ĐATN. Đó có thể là một loạt các vấn đề khó khăn mà sinh viên đã từng bước giải quyết được, là giải thuật cho một bài toán cụ thể, là giải pháp tổng quát cho một lớp bài toán, hoặc là mô hình/kiến trúc hữu hiệu nào đó được sinh viên thiết kế.

Chương này **là cơ sở quan trọng** để các thầy cô đánh giá sinh viên. Vì vậy, sinh viên cần phát huy tính sáng tạo, khả năng phân tích, phản biện, lập luận, tổng quát hóa vấn đề và tập trung viết cho thật tốt. Mỗi giải pháp hoặc đóng góp của sinh viên cần được trình bày trong một mục độc lập bao gồm ba mục con: (i) dẫn dắt/giới thiệu về bài toán/vấn đề, (ii) giải pháp, và (iii) kết quả đạt được (nếu có).

Sinh viên lưu ý **không trình bày lặp lại nội dung**. Những nội dung đã trình bày chi tiết trong các chương trước không được trình bày lại trong chương này. Vì vậy, với nội dung hay, mang tính đóng góp/giải pháp, sinh viên chỉ nên tóm lược/mô tả sơ bộ trong các chương trước, đồng thời tạo tham chiếu chéo tới đề mục tương ứng trong Chương 5 này. Chi tiết thông tin về đóng góp/giải pháp được trình bày trong mục đó.

Ví dụ, trong Chương 4, sinh viên có thiết kế được kiến trúc đáng lưu ý gì đó, là sự kết hợp của các kiến trúc MVC, MVP, SOA, v.v. Khi đó, sinh viên sẽ chỉ mô tả ngắn gọn kiến trúc đó ở Chương 4, rồi thêm các câu có dạng: “Chi tiết về kiến trúc này sẽ được trình bày trong phần 5.1”.

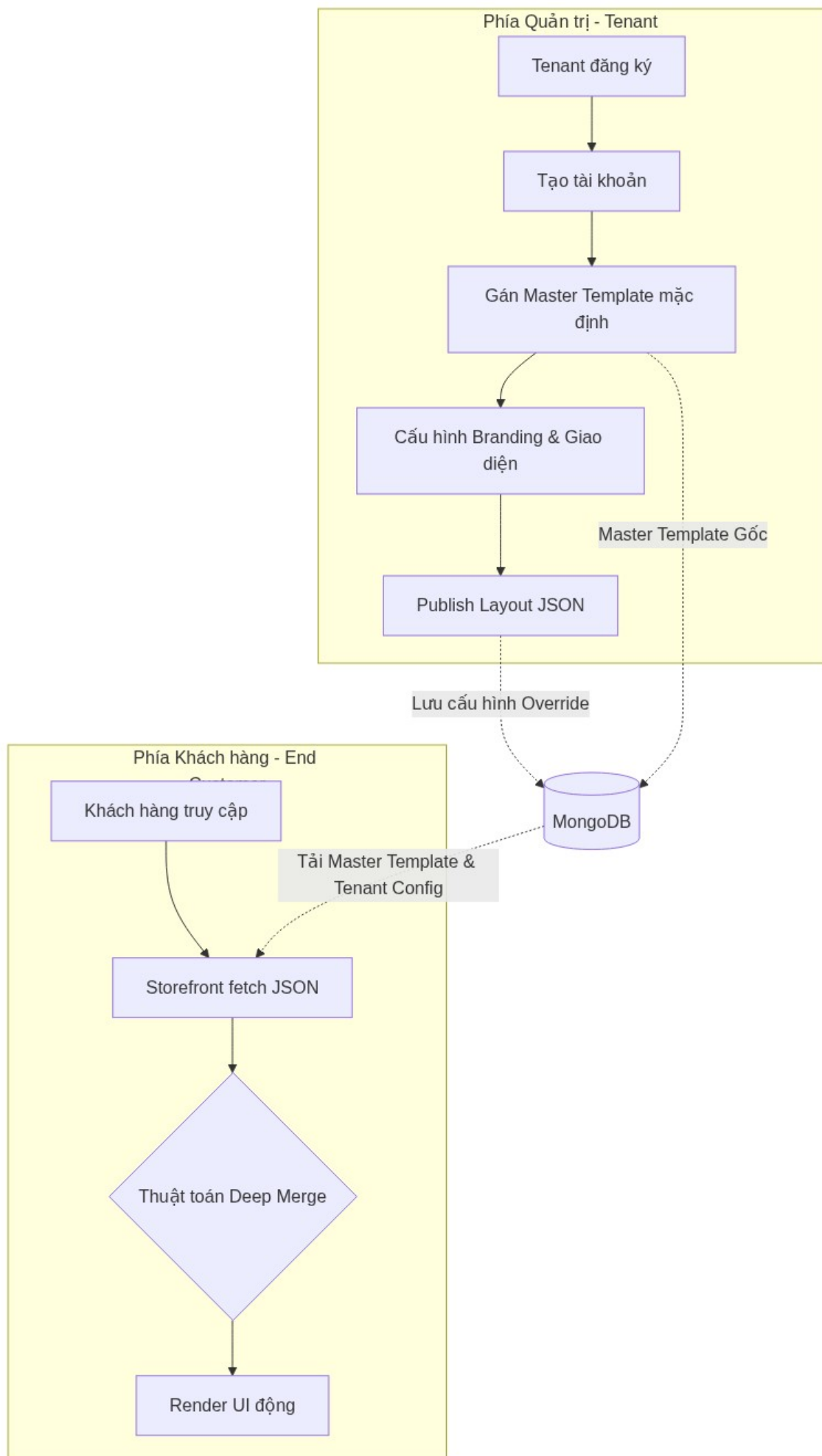
5.1 Thuật toán Deep Merge trong kiến trúc Zero-file

Thuật toán Deep Merge đóng vai trò cốt lõi trong việc hiện thực hóa kiến trúc Zero-file của nền tảng ShopVolo v2. Thay vì lưu trữ toàn bộ cấu trúc giao diện cho từng cửa hàng (tenant), hệ thống chỉ lưu trữ các bộ khung (Master Template) mặc định và các cấu hình tùy biến (Tenant Config) riêng biệt của từng cửa hàng.

Khi khách hàng truy cập vào mặt tiền cửa hàng (Storefront), hệ thống sẽ tiến hành tải cả hai tệp cấu hình JSON này và thực hiện thuật toán Deep Merge (hợp nhất đệ quy sâu). Quá trình này giúp gộp các thuộc tính tùy chỉnh từ Tenant Config đè lên các thuộc tính mặc định của Master Template, tạo ra một giao diện hoàn chỉnh và nhất quán mà không làm mất đi cấu trúc gốc [1].

¹Trong trường hợp phần này dưới 5 trang thì sinh viên nên gộp vào phần kết luận, không tách ra một chương riêng rẽ nữa.

Hình 5.1 mô tả chi tiết luồng hoạt động của quá trình từ lúc chủ cửa hàng thiết lập cấu hình cho đến khi khách hàng cuối truy cập và nhận được giao diện kết xuất tự động.



Hình 5.1: Biểu đồ luồng hoạt động thuật toán Deep Merge

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Sinh viên so sánh kết quả nghiên cứu hoặc sản phẩm của mình với các nghiên cứu hoặc sản phẩm tương tự.

Sinh viên phân tích trong suốt quá trình thực hiện ĐATN, mình đã làm được gì, chưa làm được gì, các đóng góp nổi bật là gì, và tổng hợp những bài học kinh nghiệm rút ra nếu có.

6.2 Hướng phát triển

Trong phần này, sinh viên trình bày định hướng công việc trong tương lai để hoàn thiện sản phẩm hoặc nghiên cứu của mình.

Trước tiên, sinh viên trình bày các công việc cần thiết để hoàn thiện các chức năng/nhiệm vụ đã làm. Sau đó sinh viên phân tích các hướng đi mới cho phép cải thiện và nâng cấp các chức năng/nhiệm vụ đã làm.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

[19] E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

[20] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

[21] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

[22] M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

[23] A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

[24] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available:

`ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

[25] Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] Teckel, *deepmerge - A library for deep (recursive) merging of Javascript objects*. **urlseen** 04 june 2026. **url:** <https://github.com/TehShrike/deepmerge>
- [2] VECOM, *Báo cáo Chỉ số Thương mại điện tử Việt Nam 2023*. **urlseen** 15 january 2025. **url:** <https://vecom.vn/>
- [3] Shopify Inc., *Shopify Pricing Plans*. **urlseen** 15 january 2025. **url:** <https://www.shopify.com/pricing>
- [4] Automattic Inc., *WooCommerce Features*. **urlseen** 15 january 2025. **url:** <https://woocommerce.com/>
- [5] Haravan, *Nền tảng thương mại điện tử Haravan*. **urlseen** 15 january 2025. **url:** <https://www.haravan.com/>
- [6] Adobe Inc., *Adobe Commerce (Magento)*. **urlseen** 15 january 2025. **url:** <https://business.adobe.com/products/magento/magento-commerce.html>
- [7] VECOM, *Báo cáo Chỉ số Thương mại điện tử Việt Nam 2024*. **urlseen** 15 january 2025. **url:** <https://vecom.vn/>
- [8] Vercel, *Turborepo Documentation*. **urlseen** 15 january 2025. **url:** <https://turbo.build/repo/docs>
- [9] Kamil Myśliwiec et al., *NestJS Documentation*. **urlseen** 15 january 2025. **url:** <https://docs.nestjs.com>
- [10] Vercel, *Next.js Documentation*. **urlseen** 15 january 2025. **url:** <https://nextjs.org/docs>
- [11] Prisma Data Inc., *Prisma Documentation*. **urlseen** 15 january 2025. **url:** <https://www.prisma.io/docs>
- [12] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*. **urlseen** 15 january 2025. **url:** <https://www.postgresql.org/docs/16>
- [13] MongoDB Inc., *MongoDB Manual*. **urlseen** 15 january 2025. **url:** <https://www.mongodb.com/docs/manual>
- [14] Redis Ltd., *Redis Documentation*. **urlseen** 15 january 2025. **url:** <https://redis.io/docs>
- [15] MinIO Inc., *MinIO Documentation*. **urlseen** 15 january 2025. **url:** <https://min.io/docs/minio/linux/index.html>
- [16] Better Auth Team, *Better Auth Documentation*. **urlseen** 15 january 2025. **url:** <https://www.better-auth.com/docs>

- [17] The Linux Foundation, *Kubernetes Documentation*. **urlseen** 15 **january** 2025. **url:** <https://kubernetes.io/docs>
- [18] Docker Inc., *Docker Documentation*. **urlseen** 15 **january** 2025. **url:** <https://docs.docker.com>
- [19] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial intelligence*, **jourvol** 63, **number** 1-2, **pages** 341–385, 1993.
- [20] L. L. Peterson **and** B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.
- [21] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.
- [22] M. Poesio **and** B. Di Eugenio, “Discourse structure and anaphoric accessibility,” *in* *ESSLLI workshop on information structure, discourse structure and discourse semantics, Copenhagen, Denmark* 2001, **pages** 129–143.
- [23] A. Knott, “A data-driven methodology for motivating a set of coherence relations,” phdthesis, The University of Edinburgh, UK, 1996.
- [24] T. Berners-Lee, *Hypertext Transfer Protocol (HTTP)*. **urlseen** 30 **september** 2010. **url:** <ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z>
- [25] Princeton University, *WordNet*. **urlseen** 30 **september** 2010. **url:** <http://www.cogsci.princeton.edu/~wn/index.shtml>

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

- Đối với kỹ sư chính quy:
 - Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - Từ K62 trở về sau: Ngành Khoa học máy tính
- Đối với cử nhân:
 - Ngành Công nghệ thông tin
- Đối với chương trình EliteTech:
 - Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - Chương trình ICT Global: Ngành Information Technology
 - Chương trình DS&AI: Ngành Khoa học dữ liệu

A.2 Đánh dấu (bullet) và đánh số (numering)

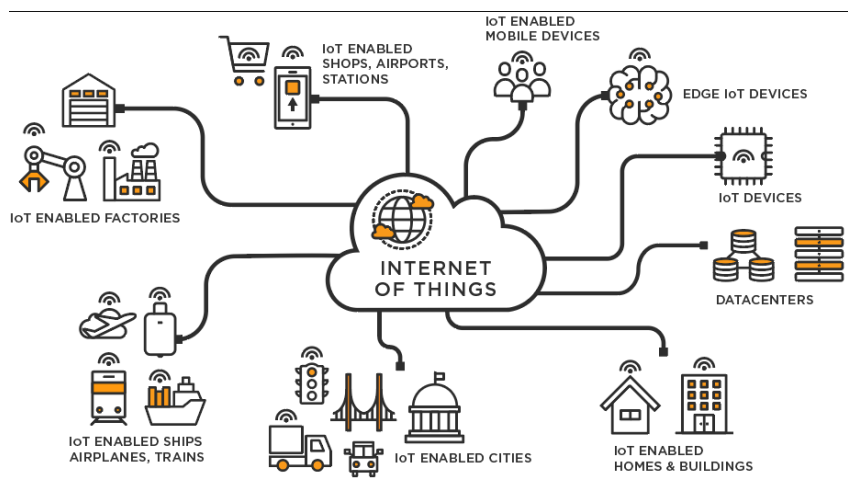
Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```

- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{enumerate}
```

Hình A.1: Internet vạn vật

`\end{enumerate}`

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong LaTeX khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau **scott2013sdn**. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau **ashton2009internet**. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

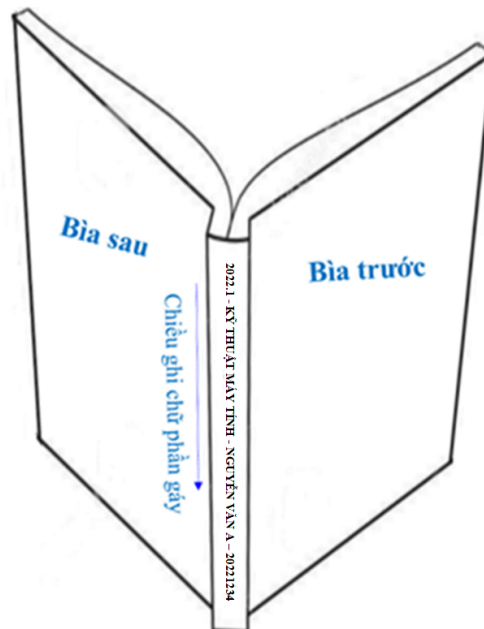
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Quy cách đóng quyển đồ án



Hình A.3: Quy cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Thống kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...